AI VIET NAM
@aivietnam.edu.vn

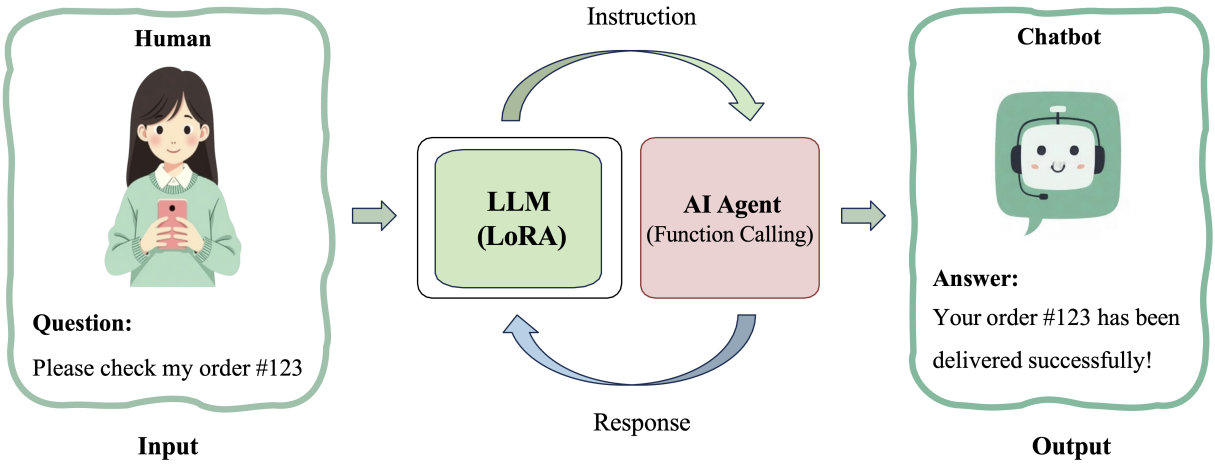
AI VIET NAM – AI COURSE 2025

AI Agents
Tool Callings for AI Agent

Nguyễn Quốc Thái Đinh Quang Vinh

I. Giới thiệu

Trong quá trình phát triển các mô hình ngôn ngữ lớn (LLMs), chúng ta ngày càng kỳ vọng nhiều hơn ở mô hình, không chỉ ở khả năng hiểu và sinh ngôn ngữ, mà còn ở việc hỗ trợ giải quyết các tác vụ thực tế. Tuy vậy, sẽ có những lúc mô hình gặp giới hạn: nó không thể tự truy cập dữ liệu thời gian thực, thực hiện tính toán chính xác, hay tương tác với hệ thống bên ngoài chỉ bằng những gì đã học. Khi đó, **Tool Calling** đóng vai trò như một cầu nối quan trọng giúp mô hình *thật sự hành động và phản hồi dựa trên thực tại.*



Hình 1: Minh họa bài toán QA sử dụng chatbot và AI Agent.

Khi người dùng gửi một câu hỏi, mô hình ngôn ngữ không trả lời ngay lập tức. Trước hết mô hình phân tích yêu cầu, sau đó quyết định có cần gọi đến công cụ bên ngoài hay không (chẳng hạn API kiểm tra đơn hàng, dịch vụ thời tiết, công cụ tìm kiếm). Sau khi nhận kết quả từ công cụ, mô hình mới sinh ra phản hồi cuối cùng dựa trên dữ liệu đã cập nhật. Chu trình hoạt động: **Suy nghĩ → Gọi công cụ → Quan sát kết quả → Trả lời**.

Trong tài liệu này, chúng ta tìm hiểu về các ví dụ thực hành đại diện cho các kiểu tool calling phổ biến nhất, từ built-in tools, prompting đơn giản, đến tool calling sử dụng langchain và tool calling fine-tuning.

Bài viết có bố cục như sau:

- **Phần I:** Giới thiệu nội dung bài viết.
- **Phần II:** Giới thiệu về Tools/Functions Calling trong LLMs (kèm 3 ví dụ: Built-in Tools, LLM Router, LangChain).
- **Phần III:** Cải thiện khả năng Tool Calling thông qua Instruction Fine-tuning.
- **Phần IV:** Thực hiện tác vụ Computer Vision thông qua Tool Calling.
- **Phần V:** Câu hỏi trắc nghiệm.
- **Phần VI:** Tài liệu tham khảo.
- **Phần VII:** Phụ lục.

Mục lục

I. Giới thiệu **1**

II. Function/Tool Calling trong LLMs **4**

II.1. Tool Calling Built-in với Llama 3 10

II.2. Tool Calling với thư viện LangChain 17

III. Cải thiện khả năng gọi tool của mô hình với Instruction Fine-tuning . . . **22**

III.1. Bước 1 – Tải và import các thư viện cần thiết 23

III.2. Bước 2 – Cấu hình môi trường (tránh lỗi Triton compile) 23

III.3. Bước 3 – Import thư viện 23

III.4. Bước 4 – Khai báo mô hình 24

III.5. Bước 5 – Khai báo cài đặt và áp dụng LoRA vào mô hình 24

III.6. Bước 6 – Tải và xử lý bộ dữ liệu Hermes 24

III.7. Bước 7 – Khai báo cài đặt huấn luyện và thực hiện huấn luyện 25

III.8. Bước 8 – Lưu mô hình đã huấn luyện 26

III.9. Bước 9 – Inference với mô hình đã fine-tune 26

IV. Các tác vụ xử lý ảnh thông qua Tool Calling **29**

IV.1. Bước 1 – Tải các thư viện cần thiết 30

IV.2. Bước 2 – Import các thư viện và chọn device 30

IV.3. Bước 3 – Định nghĩa Tools (detection + segmentation) 30

IV.4. Bước 4 – Định nghĩa danh sách tools (JSON Schema) 32

IV.5. Bước 5 – Định nghĩa system prompt 33

IV.6. Bước 6 – Khai báo mô hình LLM 34

IV.7. Bước 7 – Pipeline thực thi tool call 34

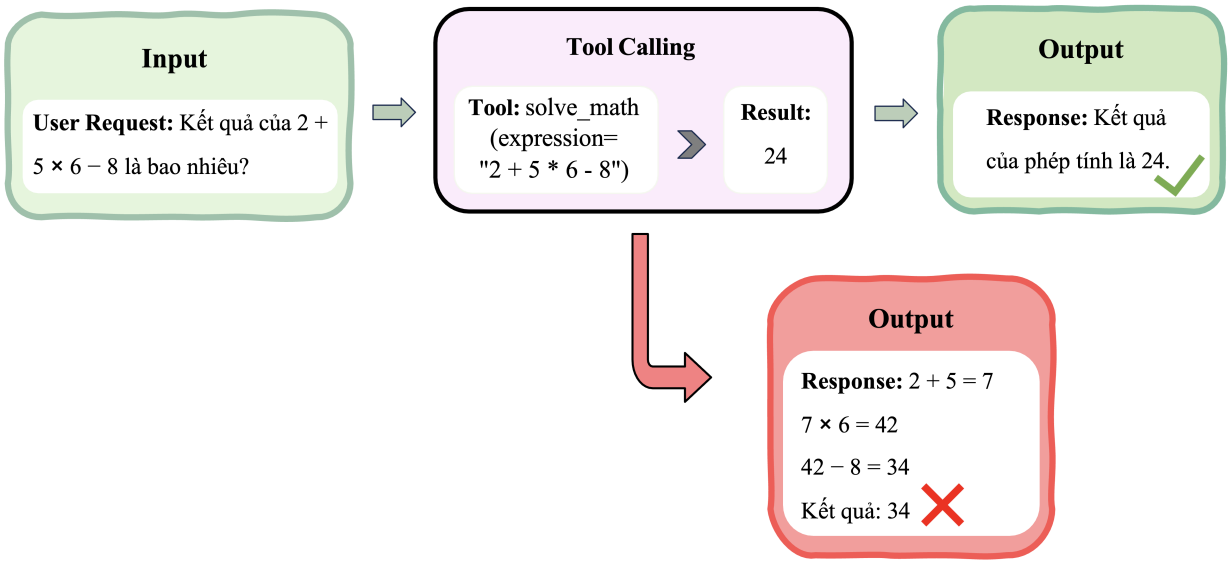
V. Câu hỏi trắc nghiệm **38**

VI. Tài liệu tham khảo **41**

Phụ lục **42**

II. Function/Tool Calling trong LLMs

Tool Calling là gì? Tool Calling là khả năng để một mô hình ngôn ngữ lớn (LLM) gọi và thực thi một công cụ hoặc hàm bên ngoài trong quá trình phản hồi người dùng. Điểm mấu chốt không nằm ở việc mô hình mô phỏng câu trả lời, mà ở việc mô hình *chủ động đề xuất hành động gọi công cụ* dựa trên khả năng suy luận và kiến thức hiện có.



Hình 2: Minh hoạ quá trình Tool Calling: với phép tính $2 + 5 \times 6 - 8$, nếu LLM tự tính có thể ra sai (34); nhưng nếu gọi tool `solve_math`, kết quả luôn đúng (24).

Khác với việc đoán đáp án, mô hình có thể đưa ra suy nghĩ như: "Tôi không thể tính phép toán này, nhưng tôi biết công cụ nào có thể". Chẳng hạn, thay vì trả lời ngay phép tính $2+5\times6-8$, mô hình tạo ra một lệnh gọi công cụ tính toán:

Lệnh gọi công cụ tính toán

```
# Gọi công cụ tính toán
Tool: solve_math(expression="2 + 5 * 6 - 8")
# Kết quả: 24
```

Sau khi công cụ thực thi, kết quả (24) được trả về và mô hình phản hồi: "Kết quả của phép tính là 24". Cách tiếp cận này giúp LLM trở nên giống như một trợ lý thực sự: biết cách khai thác công cụ để tìm câu trả lời.

Vậy Function Calling là gì và có gì khác biệt? Tool Calling là khái niệm chung chỉ khả năng của mô hình khi tương tác với các công cụ bên ngoài. Function Calling là một cách cụ thể để thực hiện điều đó — đặc biệt phổ biến trong hệ sinh thái OpenAI. Với Function Calling, các hàm mà mô hình có thể gọi sẽ được mô tả theo định dạng **JSON Schema**. Mô hình "nhìn thấy" tên hàm, các tham số cần truyền vào và kiểu dữ liệu tương ứng, từ đó gọi đúng hàm với đúng thông tin:

Function schema (JSON)

```json
{
  "name": "get_weather",
  "description": "Lấy thông tin thời tiết",
  "parameters": {
    "type": "object",
    "properties": {
      "location": { "type": "string" }
    },
    "required": ["location"]
  }
}
```

Không chỉ dừng lại ở việc hiểu hàm, mô hình còn có thể tự suy luận ra hành động cần làm: khi nào nên gọi hàm, cần truyền tham số nào, và xử lý kết quả trả về ra sao. Toàn bộ quá trình gồm ba bước chính:

- **Thought (Suy nghĩ):** phân tích yêu cầu đầu vào.
- **Act (Hành động):** sinh lời gọi hàm.
- **Observe (Quan sát):** nhận và xử lý kết quả trả về.

Function Calling tạo ra một "ngôn ngữ hành động" có cấu trúc, giúp LLM hành xử như một thành phần biết tương tác logic trong hệ thống lớn. Lưu ý: **mô hình không trực tiếp thực thi hàm**. Nó chỉ sinh lệnh gọi hàm dưới dạng JSON; việc thực thi thuộc về hệ thống bên ngoài, sau đó kết quả mới được mô hình dùng để tạo phản hồi hoàn chỉnh.

Khi nào nên dùng Tool/Function Calling? Tool/Function Calling thường được sử dụng khi LLM phải giải quyết các tác vụ mà bản thân nó không thể hoặc rất khó tự giải quyết chỉ bằng ngôn ngữ tự nhiên:

- Truy vấn cơ sở dữ liệu hoặc hệ thống quản lý thông tin nội bộ.
- Tìm kiếm thông tin mới nhất (giá cổ phiếu, thời tiết, tin tức).
- Thực hiện phép toán phức tạp hoặc thao tác logic.
- Tương tác với các API hoặc hệ thống phần mềm khác.
- Triển khai công việc chuyên môn: phân tích y học, tìm kiếm học thuật, truy xuất bản đồ.

Các loại công cụ trong Tool Calling. Dựa trên cách tích hợp và ngữ cảnh sử dụng, các công cụ chia thành ba loại chính:

- **Built-in Tool**: công cụ tích hợp sẵn trong mô hình như Wolfram Alpha, Python, Brave Search trong Llama 3.
- **JSON-based Tool**: thường gặp trong hệ sinh thái OpenAI, function được định nghĩa bằng JSON schema rõ ràng.

- **Custom Tool**: do người dùng định nghĩa và kiểm soát, ví dụ các hàm Python tuỳ biến hoặc endpoint API riêng.

Ví dụ về việc sử dụng công cụ toán học Wolfram để giải một biểu thức:

Wolfram tool call

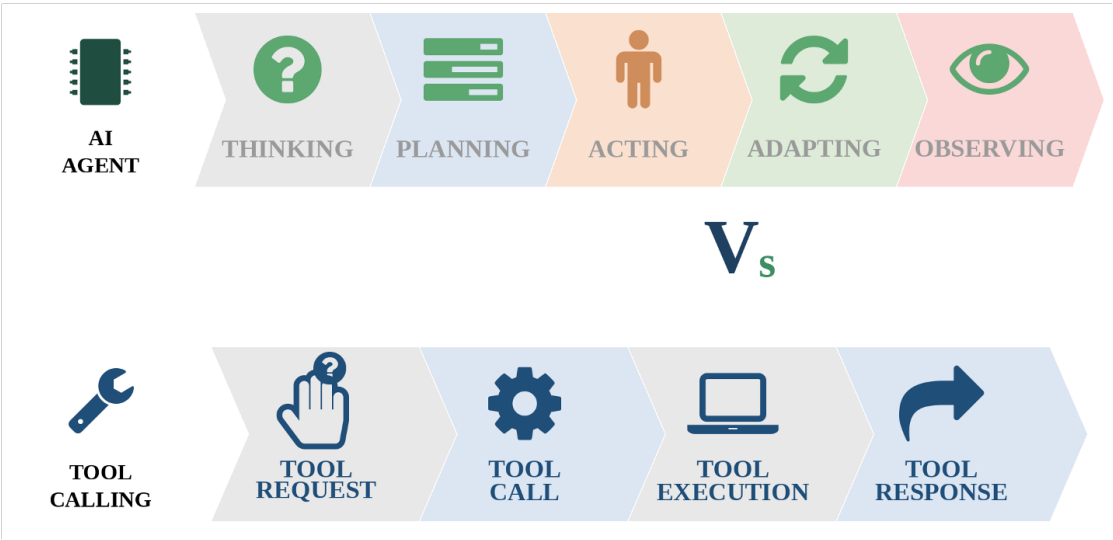
```
# Lời gọi công cụ Wolfram
Tool: wolfram_alpha.calculate("2*y^2 + x^2")
# Trả về biểu thức được rút gọn hoặc hình ảnh biểu diễn
```

Mỗi loại công cụ phù hợp với một mục tiêu và môi trường triển khai khác nhau.

Tool Calling có phải là AI Agent? Không hoàn toàn. Khái niệm *agent* là một trong những nền tảng quan trọng nhất trong lĩnh vực Trí tuệ nhân tạo. Trong giáo trình kinh điển *Artificial Intelligence: A Modern Approach*, Stuart Russell và Peter Norvig định nghĩa:

> *"An agent is anything that can be viewed as perceiving its environment through sensors and acting upon that environment through actuators. The agent function maps percept histories to actions."*
> — Stuart Russell & Peter Norvig, *AI: A Modern Approach*, 3rd Edition, Chapter 2, p. 34.

(Tạm dịch): *"Một agent là bất kỳ thực thể nào có thể được xem là có khả năng cảm nhận môi trường xung quanh thông qua các cảm biến, và đưa ra hành động tác động lại lên môi trường thông qua các bộ phận điều khiển. Hàm agent ánh xạ lịch sử nhận thức thành các hành động."*



Hình 3: So sánh chu trình AI Agent (5 bước Thinking/Planning/Acting/Adapting/Observing) và Tool Calling (4 bước Tool Request/Tool Call/Tool Execution/Tool Response).

Theo định nghĩa trên, **AI Agent** là hệ thống có khả năng cảm nhận môi trường, lập kế hoạch và thực hiện hành động dựa trên thông tin nhận được. Chu trình hành vi của một agent là vòng lặp liên tục:

Observing → Thinking/Planning → Acting → Adapting → Observing

Một agent cần có:

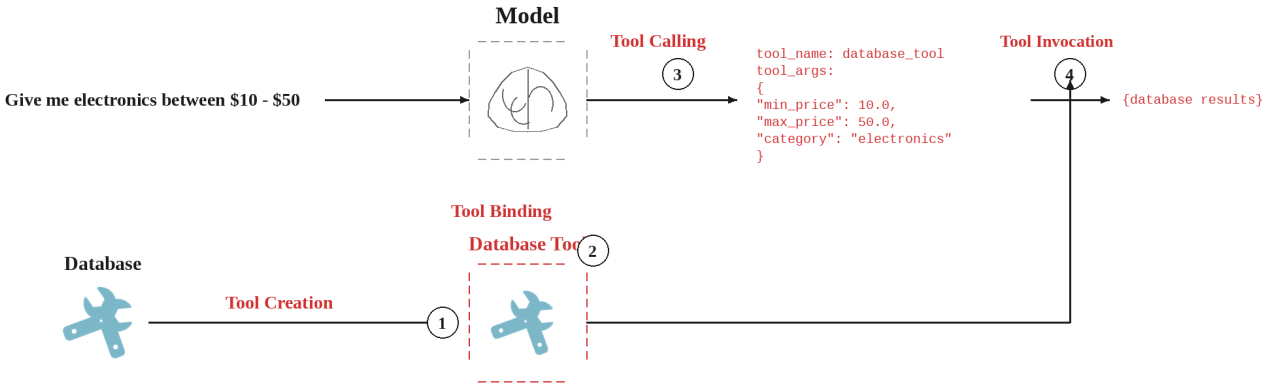
- **Quan sát (observing)**: thu nhận thông tin từ môi trường.
- **Tư duy / Lập kế hoạch (thinking/planning)**: phân tích bối cảnh và xây dựng chiến lược.
- **Hành động (acting)**: thực thi quyết định.
- **Điều chỉnh (adapting)**: điều chỉnh hành vi dựa trên kết quả.

Một mô hình ngôn ngữ có thể biết cách gọi công cụ (tool calling), nhưng nếu thiếu các bước lập kế hoạch, quan sát phản hồi, hoặc điều chỉnh hành vi, thì nó vẫn chỉ là một hệ thống phản hồi theo lượt (*turn-based reactive system*), chưa phải *AI Agent* thực thụ. Ngược lại, một agent có thể hoạt động mà không cần gọi công cụ, miễn là nó có chu trình hành vi đầy đủ. Do đó:

- **Tool Calling** là một *kỹ năng* (gọi đúng công cụ vào đúng thời điểm).
- **AI Agent** là một *hệ thống hành vi tự chủ* — có khả năng quan sát, suy luận, hành động và học hỏi từ môi trường.

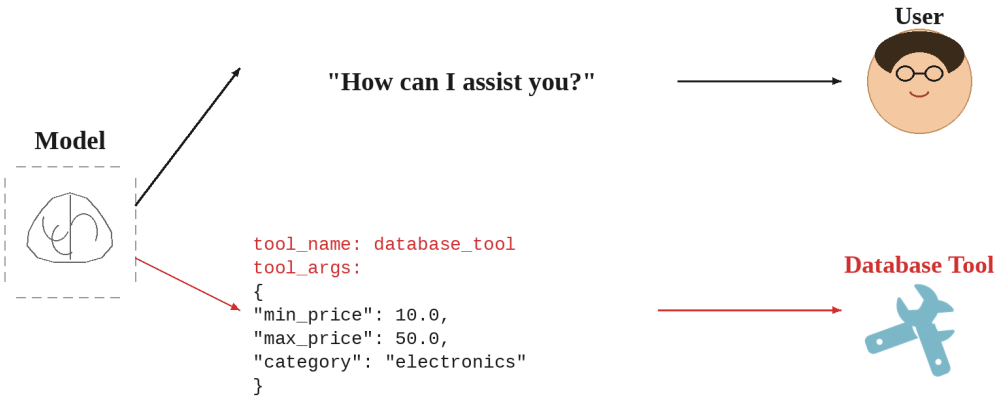
Phân biệt rõ hai khái niệm này giúp tránh nhầm lẫn giữa "một mô hình biết sử dụng công cụ" và "một hệ thống có khả năng tư duy độc lập, hành động có chiến lược và thích nghi với thay đổi".

Tool Calling cũng có một quy trình 4 bước rõ ràng từ tạo tool đến gọi tool:



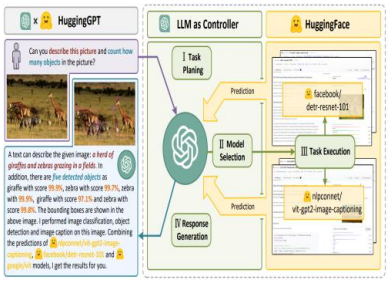
Hình 4: Tool Calling Workflow: **(1)** Tool Creation — đăng ký tool, **(2)** Tool Binding — gắn tool vào model, **(3)** Tool Calling — model sinh JSON call, **(4)** Tool Invocation — client thực thi và trả kết quả về.

Sự khác biệt giữa nhánh *trả lời tự nhiên* và nhánh *gọi tool* được minh hoạ rõ qua hình sau — mô hình có thể chọn một trong hai dựa trên nội dung câu hỏi:



Hình 5: Model chọn nhánh: nếu hỏi xã giao thì trả lời tự nhiên "How can I assist you?"; nếu cần dữ liệu cụ thể, model sinh tool call dạng `tool_name` + `tool_args`.

Sau khi đã tìm hiểu các khái niệm Tool Calling, Function Calling và AI Agent, ta điểm qua hành trình phát triển của công nghệ này:



1

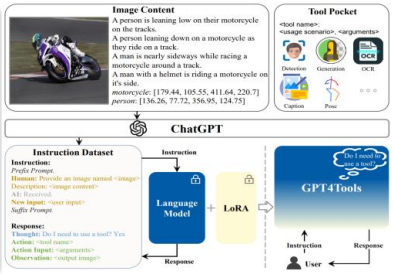
HuggingGPT 03/2023

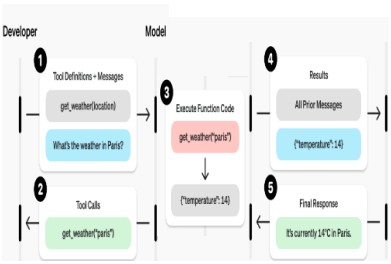
LLM lần đầu đóng vai "bộ não" điều phối: phân tích yêu cầu, lập kế hoạch, và kích hoạt các mô hình AI chuyên sâu (text, image, speech...) từ Hugging Face. Đây là nền tảng cho việc tích hợp công cụ (tool use) vào LLM.

GPT4Tools 05/2023 LLMs as Tool Makers

Phát triển từ ý tưởng trên, GPT-4 không chỉ gọi công cụ (tool calling) mà còn tự động tạo lệnh (function generation) dựa trên mô tả. Bước nhảy vọt này giúp LLM chuyển từ thao tác thủ công sang tự học cách ứng dụng công cụ.

2





3

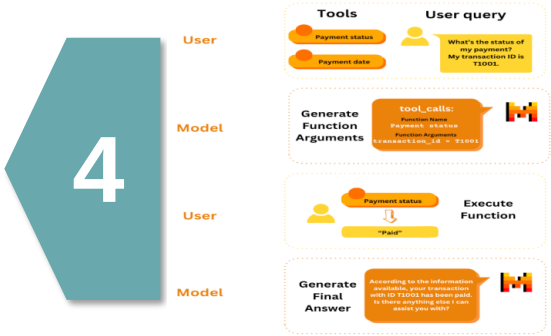
OpenAI Function Calling API 06/2023

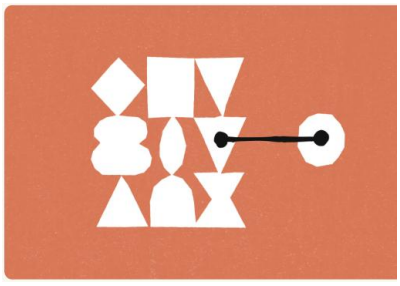
Kế thừa xu hướng, OpenAI ra mắt API cho phép LLM sinh lệnh gọi hàm (function calls) dạng JSON. Công cụ này kết nối mô hình với hệ thống backend, cơ sở dữ liệu... một cách chính xác và an toàn, mở đường cho tích hợp đa nền tảng.

Function Calling in AI Agents (Mistral 7B) 10/2023

Tiến thêm một bước, Function Calling trở thành phần cốt lõi trong vòng lặp agent (agent loop): ví dụ Mistral 7B thực hiện chuỗi suy nghĩ → chọn công cụ → thực thi → cải thiện (*planning* → *acting* → *observing* → *adapting*). Quy trình cho phép xử lý tác vụ đa bước (multi-step tasks) trong môi trường phản hồi.

4





5

Model Context Protocol (MCP) – Anthropic 11/2024

Đến cuối năm 2024, MCP cho phép nhiều LLM chia sẻ ngữ cảnh (context sharing) và phối hợp như một hệ đa agent (multi-agent). Đây là bước tiến lớn cho các hệ thống AI phức tạp và linh hoạt hơn.

Hình 6: Khảo sát các mốc phát triển chính của Tool/Function Calling trong LLMs (2023–2024).

II.1. Tool Calling Built-in với Llama 3

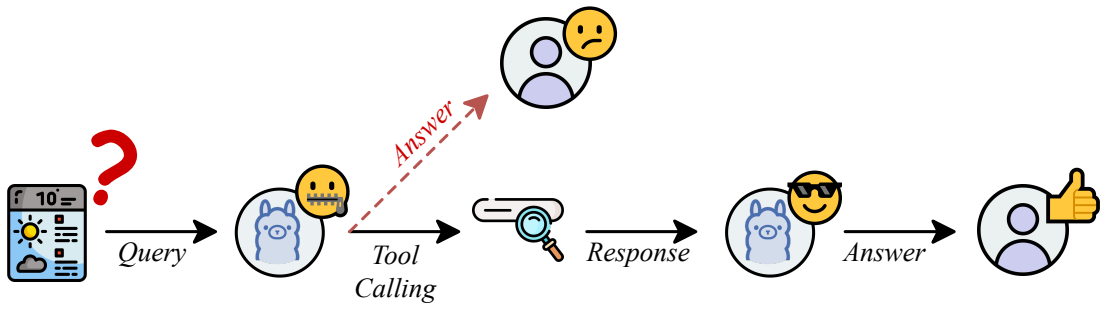
Tool Calling là tính năng được tích hợp sẵn trong `Llama 3`, giúp mô hình chủ động sử dụng các công cụ bổ trợ để hoàn thành những tác vụ mà bản thân mô hình gặp khó khăn. Có ba hình thức Tool Calling chính:

- **Built-in Tools** (công cụ tích hợp sẵn như Brave Search và Wolfram Alpha).
- **JSON-based Tool Calling** (gọi công cụ dựa trên cấu trúc JSON).
- **User-defined Custom Tool Calling** (gọi công cụ do người dùng tự định nghĩa).

Mục này minh hoạ **built-in tools**. Cụ thể, các phiên bản `Llama 3.1-8B/70B/405B` có khả năng sử dụng trực tiếp hai công cụ tích hợp dựa trên Python:

- **Brave Search**: thực hiện tìm kiếm trên web.
- **Wolfram Alpha**: thực hiện các phép toán phức tạp.

Đầu vào của mô hình là một câu hỏi truy vấn thông tin (vd "Thời tiết ở Hồ Chí Minh hôm nay như thế nào?") và mô hình tạo ra câu trả lời bằng cách tổng hợp thông tin được cung cấp từ công cụ.



Hình 7: Minh hoạ quá trình truy vấn thông tin về thời tiết. Trường hợp mô hình không sử dụng công cụ được tô màu đỏ và thường trả về kết quả không như kỳ vọng.

Để kích hoạt việc gọi các công cụ tích hợp, ta sử dụng một đoạn khai báo đặc biệt gọi là *System Prompt*:

System Prompt cho Llama 3 built-in

```
<|begin_of_text|><|start_header_id|>system<|end_header_id|>

Environment: ipython
Tools: brave_search, wolfram_alpha
Cutting Knowledge Date: December 2023
Today Date: 23 July 2024

You are a helpful assistant<|eot_id|>
```

```
<|start_header_id|>user<|end_header_id|>

What is the current weather in Menlo Park, California?<|eot_id|>
<|start_header_id|>assistant<|end_header_id|>
```

Một số điểm cần lưu ý về cách thức mô hình thực hiện Tool Calling:

- Việc khai báo `Environment: ipython` tự động kích hoạt tính năng Code Interpreter mà không cần ghi rõ trong mục `Tools`.
- Khi trả lời, nội dung do mô hình tạo ra sẽ bắt đầu với một tag đặc biệt `<|python_tag|>`.
- Mô hình có thể sử dụng tag `<|eom_id|>` để biểu thị quá trình suy luận nhiều bước nhưng chưa kết thúc, khác với `<|eot_id|>` là dấu hiệu cho thấy lượt trả lời đã hoàn tất.

Các công cụ tích hợp được gọi bằng cú pháp Python cụ thể như sau:

Cú pháp gọi built-in tools

```
# Brave Search
<|python_tag|>
brave_search.call(query="...")
<|eom_id|>

# Wolfram Alpha
<|python_tag|>
wolfram_alpha.call(query="...")
<|eom_id|>
```

Để đơn giản hoá, việc triển khai minh hoạ này sử dụng mô hình `Llama 3.1-8B-Instant` (cho calculation) và `Llama 3.3-70B-Versatile` (cho search) thông qua API của Groq. Công cụ tìm kiếm Brave Search hoặc `ddgs` (DuckDuckGo Search); công cụ Wolfram Alpha thực hiện tính toán.

II.1.1. Bước 1 – Thiết lập môi trường, mô hình và công cụ

Cài đặt các thư viện cần thiết:

Cài đặt thư viện

```
!pip install -q ddgs groq wolframalpha
```

Sau đó import và khởi tạo client:

Import + khởi tạo client

```
import os
import re
```

```
import json
import asyncio
import aiohttp
import requests
import nest_asyncio
import urllib.parse
from groq import Groq
from google.colab import userdata
from ddgs import DDGS
import warnings
warnings.filterwarnings("ignore", category=DeprecationWarning)

nest_asyncio.apply()

os.environ["GROQ_API_KEY"] = "YOUR_GROQ_API_KEY"
APP_ID = "YOUR_WOLFRAM_APP_ID"

client = Groq()
```

Trong đó, các bước lấy **API Key** của Groq:

1. Đăng ký/đăng nhập tại console.groq.com.
2. Trên thanh điều hướng (góc trên bên phải), chọn "API Keys" và đợi chuyển trang, sau đó nhấn "Create API Key".
3. Sao chép API Key vào biến `GROQ_API_KEY` trong đoạn mã trên.

Với Wolfram Alpha, lấy **App ID**:

1. Đăng ký/đăng nhập tại developer.wolframalpha.com.
2. Chọn "Get an App ID", điền các trường Name/Description và chọn **Simple API** cho mục API, sau đó nhấn "Submit".
3. Sao chép đoạn ký tự bên dưới "App ID" (vd: `JH5XYZ-ABCDE9XU5W`) và thay thế `APP_ID` trong đoạn mã.

II.1.2. Bước 2 – Xây dựng các hàm xử lý

Sau khi thiết lập được mô hình và công cụ, để xử lý việc gọi và nhận kết quả từ API, ta cần các hàm:

- `model_response(prompt)`: gửi prompt đến mô hình và trả về phản hồi.
- `extract_query(tool_call_text)`: trích xuất tên công cụ và nội dung truy vấn từ chuỗi đầu vào. Regex được mở rộng để khớp cả `brave_search(query=...)` (không có `.call`) và `brave_search.call(query=...)`, vì model có thể sinh hai biến thể này.
- `format_search_results(results)`: định dạng danh sách kết quả tìm kiếm thành chuỗi văn bản dễ đọc.

- `wolfram_short_answer(query)`: gửi truy vấn đến Wolfram Alpha API và trả về câu trả lời ngắn (xử lý bất đồng bộ).

Các hàm xử lý

```python
def model_response(prompt):
  response = client.chat.completions.create(
      messages=[
          {
              "role": "user",
              "content": prompt,
          }
      ],
    #   model="llama-3.3-70b-versatile",  # for search
      model="llama-3.1-8b-instant",  # for calculation
  )

  return response

def extract_query(tool_call_text):
    brave_match = re.search(
        r'brave_search(?:\.call)?\(query="(.*?)"\)', tool_call_text)
    if brave_match:
        return "brave_search", brave_match.group(1)
    wolfram_match = re.search(
        r'wolfram_alpha(?:\.call)?\(query="(.*?)"\)', tool_call_text)
    if wolfram_match:
        return "wolfram_alpha", wolfram_match.group(1)
    return None, None

def format_search_results(results):
    lines = []
    for i, item in enumerate(results, start=1):
        title = item.get("title", "No title")
        href = item.get("href", "No link")
        body = item.get("body", "No description")
        lines.append(f"{i}. {title}\n   Description: {body}\n   Link: {href}\n")
    return "\n".join(lines)

async def wolfram_short_answer(query):
    encoded_query = urllib.parse.quote(query)
    url = f"https://api.wolframalpha.com/v1/result?i={encoded_query}&appid={APP_ID}"
    async with aiohttp.ClientSession() as session:
        async with session.get(url) as response:
            if response.status == 200:
                return await response.text()
            else:
                return f"[Error {response.status}] {await response.text()}"
```

II.1.3. Bước 3 – Xây dựng pipeline LLM Tool Calling

Quy trình Tool Calling gồm 5 bước rõ ràng:

1. Người dùng cung cấp yêu cầu (prompt) cùng với System Prompt.

2. Mô hình quyết định công cụ cần gọi.
3. Công cụ thực thi và trả kết quả về (vd Wolfram Alpha thực hiện phép tính).
4. Kết quả công cụ được gửi trở lại cho mô hình.
5. Mô hình tổng hợp và gửi phản hồi cuối cùng tới người dùng.

Toàn bộ pipeline được đóng gói trong hàm `llama_with_tool`. System prompt được viết chặt chẽ để ép mô hình chỉ trả về duy nhất một dòng tool call, không kèm giải thích, đảm bảo regex parse được đúng:

Hàm llama_with_tool

```
async def llama_with_tool(query):
    PROMPT = f"""
<|begin_of_text|><|start_header_id|>system<|end_header_id|>

Environment: ipython
Tools: brave_search, wolfram_alpha
Cutting Knowledge Date: December 2023
Today Date: 23 July 2024

You are a tool-using assistant. You have access to exactly two tools:
    1. **brave_search**  for any factual, news, weather, or current-events question
    2. **wolfram_alpha**  for any math, calculation, or symbolic-computation question

CRITICAL INSTRUCTIONS:
- For ANY question above, output EXACTLY ONE LINE in this format and NOTHING ELSE:
  <|python_tag|>brave_search.call(query="...")
  OR
  <|python_tag|>wolfram_alpha.call(query="...")
- Do NOT explain, do NOT reason, do NOT solve manually.
- Do NOT write code, do NOT show steps.
- Pick exactly ONE tool. Output the tool call line only.
<|eot_id|><|start_header_id|>user<|end_header_id|>
{query}<|eot_id|><|start_header_id|>assistant<|end_header_id|>
    """

    response = model_response(PROMPT)
    tool_call = response.choices[0].message.content.strip()
    print("=== Model Tool Call Output ===\n")
    print(tool_call)
    tool_name, query_content = extract_query(tool_call)
    print("\n=== Extracted Query Content ===\n")
    print(query_content)
    if tool_name == "brave_search":
        tool_response = DDGS().text(query_content, max_results=5)
        formatted_results = format_search_results(tool_response)
    elif tool_name == "wolfram_alpha":
        res = await wolfram_short_answer(query_content)
        formatted_results = "The exact value of the query is " + res
    else:
        formatted_results = "No tool could be identified."
    print("\n=== Tool Responses ===\n")
```

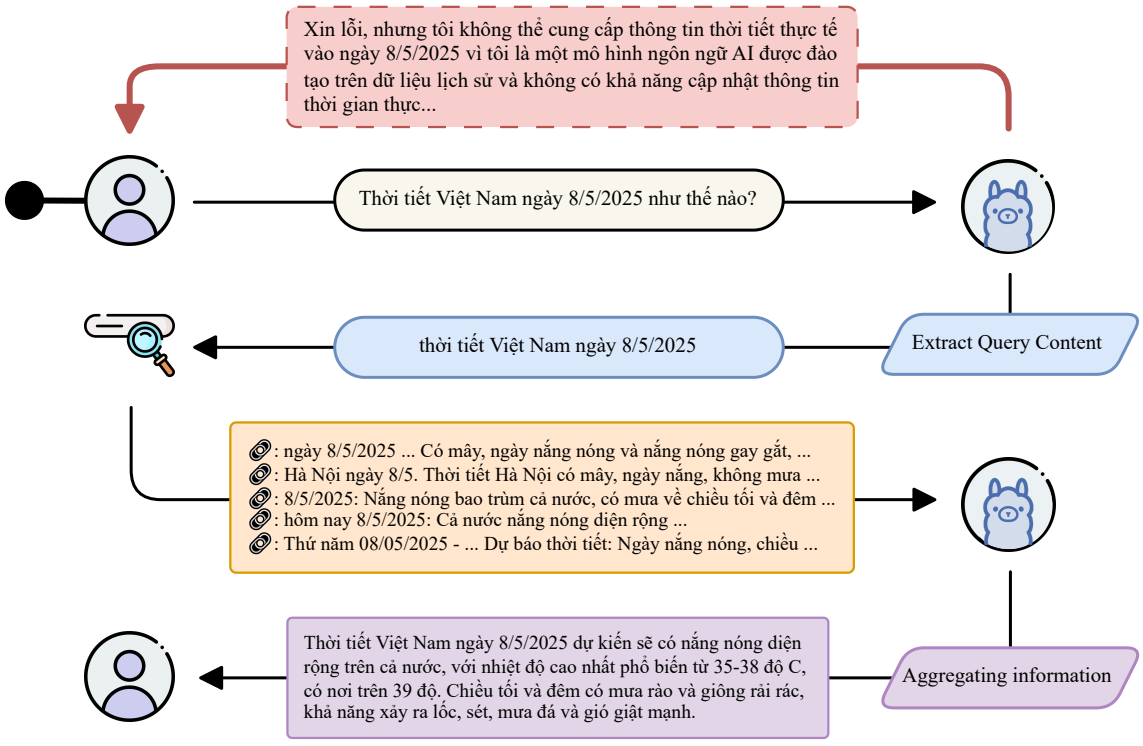
```
    print(formatted_results)
    follow_up_prompt = f"Tool responses:\n{formatted_results}\nBased on these responses
                                        , provide a concise result for this query.:
                                        {query}"
    follow_up_response = model_response(follow_up_prompt)
    print("\n=== Final Answer ===\n")
    return follow_up_response.choices[0].message.content
```

II.1.4. Bước 4 – Demo tìm kiếm thông tin thời gian thực

Với câu hỏi cần thông tin thời gian thực (vd thời tiết), mô hình sẽ tự động gọi `brave_search`:

Search demo

```
search_question = "Thời tiết Việt Nam ngày 7/5/2026 như thế nào?"
print(model_response(search_question).choices[0].message.content)
print(await llama_with_tool(query=search_question))
```



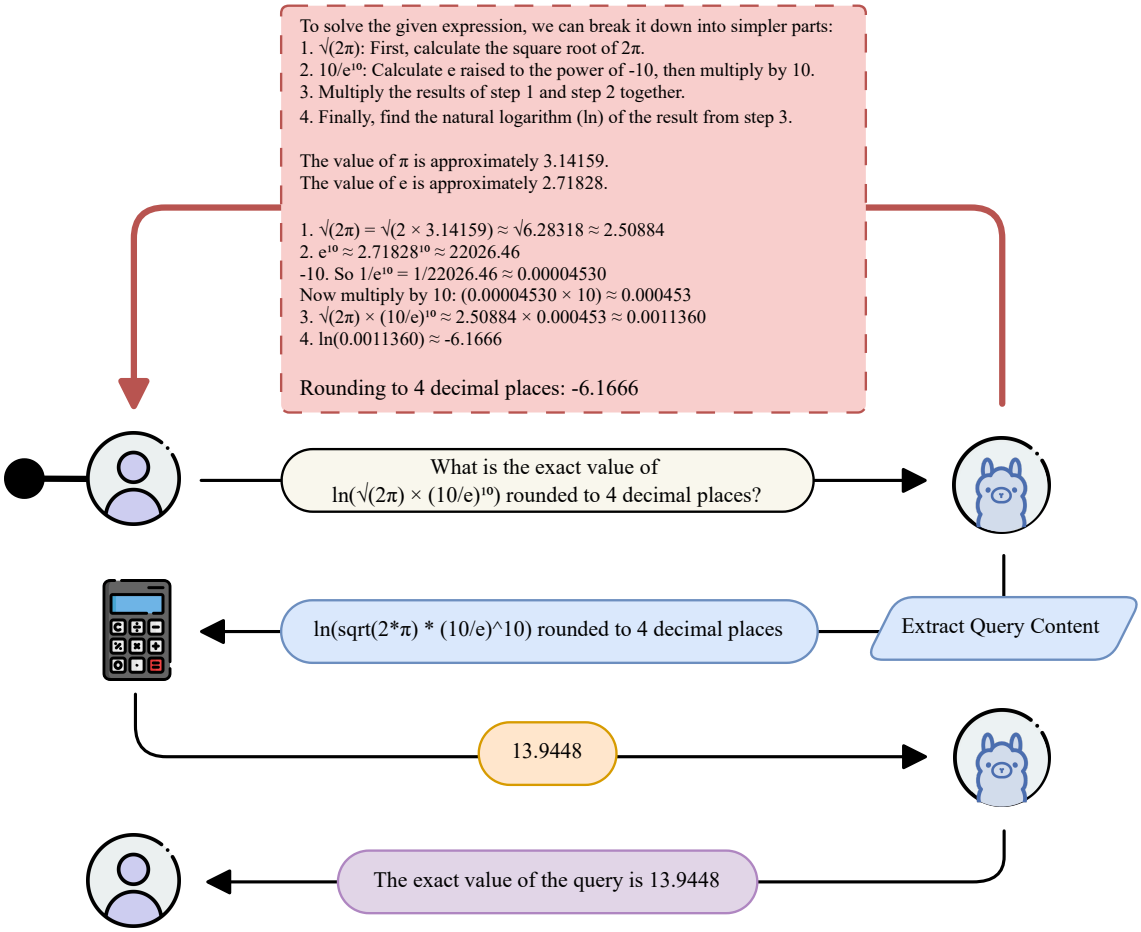
Hình 8: Quy trình xử lý và phản hồi từ Llama 3 với công cụ tìm kiếm. Đường màu đỏ là phản hồi không sử dụng công cụ; đường màu đen là khi có sử dụng.

II.1.5. Bước 5 – Demo phép tính phức tạp với Wolfram Alpha

Với câu hỏi cần phép tính chính xác (như logarit, tích phân, biểu thức symbolic), mô hình sẽ tự gọi `wolfram_alpha`:

Math demo

```
math_question = "What is the exact value of ln((2)  (10/e)) rounded to 4 decimal places ?"
print(model_response(math_question).choices[0].message.content)
print(await llama_with_tool(query=math_question))
```

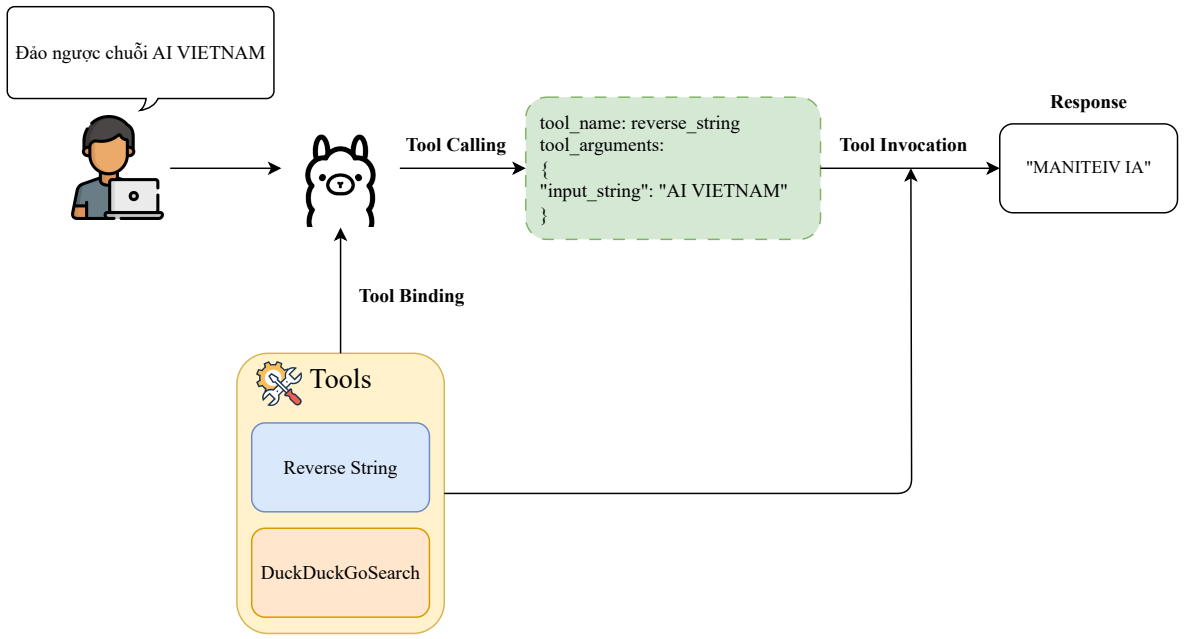


Hình 9: Quy trình xử lý và phản hồi từ Llama3.1 với công cụ tính toán phức tạp. Trong đó, phản hồi không sử dụng công cụ cho kết quả sai và ngược lại khi có dùng công cụ.

Nhờ tích hợp Tool Calling, `Llama 3` có thể giải quyết hiệu quả các vấn đề phức tạp, giúp người dùng nhận được câu trả lời chính xác và hữu ích hơn so với việc tự suy luận thuần tuý từ tham số đã học.

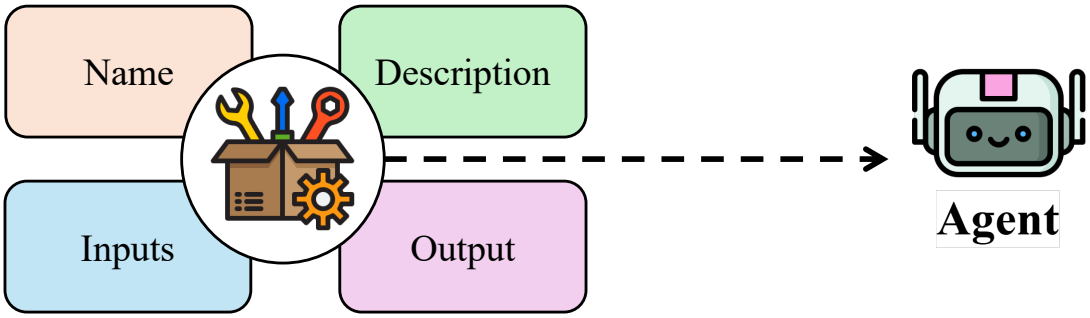
II.2. Tool Calling với thư viện LangChain

Hiện tại, việc cài đặt các tool cho mô hình lớn được hỗ trợ rất nhiều trên những thư viện chuyên dụng như LangChain, crewAI, LlamaIndex... Phần này sử dụng thư viện LangChain để minh hoạ. So với cách triển khai built-in ở phần trước, LangChain cung cấp lớp trừu tượng cao hơn: decorator `@tool` và phương thức `bind_tools()` giúp pipeline ngắn gọn và đồng nhất giữa các backend (OpenAI, Anthropic, Ollama).



Hình 10: Mô tả LLMs Tool Calling với thư viện LangChain.

Lưu ý quan trọng: ta dùng `ChatOllama` với mô hình `qwen2.5:7b`. Khi định nghĩa tool, chúng ta cần chi rõ name (tên), description (mô tả chức năng, dữ liệu đầu vào, đầu ra), inputs (dữ liệu đầu vào), outputs (thực hiện chương trình để nhận được kết quả đầu ra là gì). Việc mô tả đầy đủ giúp LLM đưa ra quyết định chính các hơn.



Hình 11: Minh hoạ một số thông tin mà Agent cần để hiểu ý nghĩa của một tool do ta tự định nghĩa.

II.2.1. Bước 1 – Cài đặt Ollama và pull mô hình

Cài Ollama vào Colab/local:

Cài đặt Ollama

```
!apt-get install -y zstd
!curl -fsSL https://ollama.com/install.sh | sh
```

Khởi động Ollama server trong nền (mặc định cổng 11434):

Start Ollama server

```
import subprocess, time, os
subprocess.run(['pkill', '-f', 'ollama'], stderr=subprocess.DEVNULL)
time.sleep(2)
subprocess.Popen(['ollama', 'serve'],
                 stdout=open('/tmp/ollama.log', 'w'),
                 stderr=subprocess.STDOUT,
                 env={**os.environ, 'OLLAMA_HOST': '0.0.0.0:11434'})
time.sleep(5)
```

Pull mô hình `qwen2.5:7b` (chỉ cần làm 1 lần, mô hình được cache lại):

Pull qwen2.5:7b

```
!ollama pull qwen2.5:7b
```

Verify Ollama API

```
!curl -s http://localhost:11434/api/tags
```

II.2.2. Bước 2 – Cài đặt thư viện LangChain

Pip install

```
!pip install -U ddgs langchain_community langchain-ollama
```

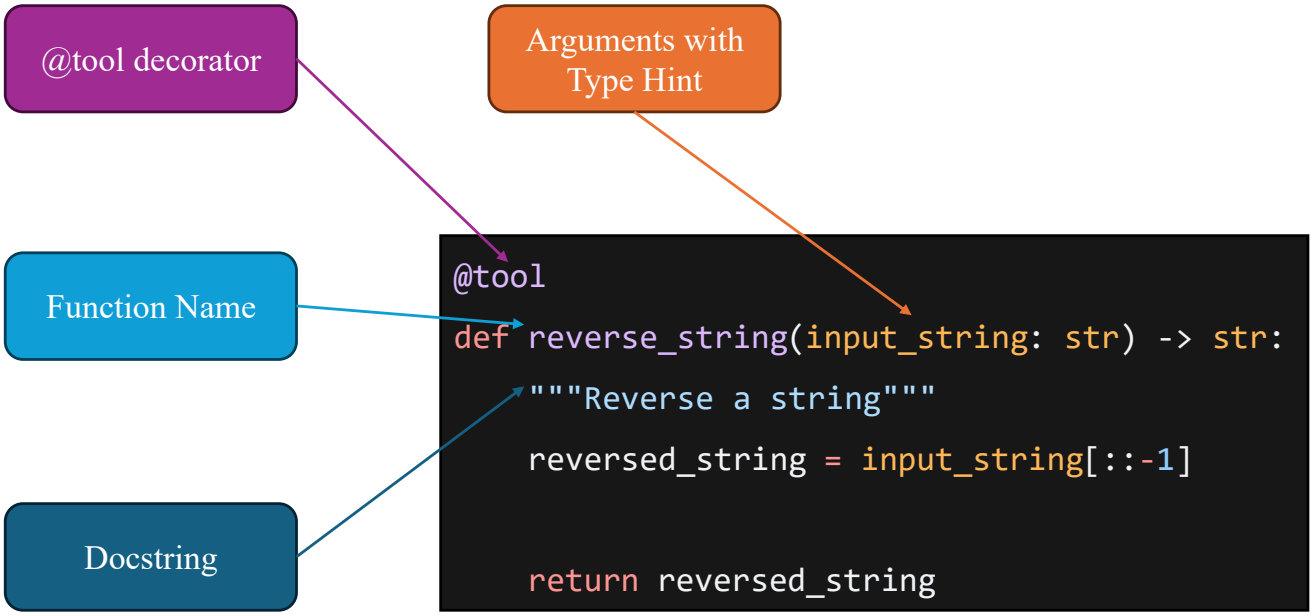
Imports

```
from langchain_core.tools import tool
from langchain_community.utilities import DuckDuckGoSearchAPIWrapper
from langchain_community.tools import DuckDuckGoSearchResults
```

II.2.3. Bước 3 – Định nghĩa các tool bằng decorator @tool

Đối với LangChain, để định nghĩa các tool ta cần khai báo hàm Python với đầy đủ các thành phần sau:

1. **@tool decorator**: để LangChain hiểu và đưa hàm Python trở thành công cụ cho LLM, ta dùng decorator `@tool`.
2. **Tên hàm**: tên hàm phải thể hiện rõ chức năng/công dụng/mục đích của công cụ.
3. **Tham số đầu vào kèm type hint**: khi định nghĩa biến đầu vào, cần cung cấp cả thông tin về kiểu dữ liệu. Mô hình dựa vào đó để gọi đúng tham số.
4. **Docstring**: một dòng mô tả ngắn cho chức năng của hàm. Đây là thứ giúp mô hình hiểu công cụ ta muốn nó sử dụng.



Hình 12: Minh hoạ các thành phần cần thiết để định nghĩa một tool trong LangChain.

Chúng ta định nghĩa ba tool: `reverse_string` (đảo ngược chuỗi), `search` (tìm kiếm web), và `multiply` (nhân hai số nguyên):

reverse_string

```python
@tool
def reverse_string(input_string: str) -> str:
    """Reverse a string"""
    reversed_string = input_string[::-1]
    return reversed_string
reversed_string = reverse_string.invoke("AI VIET NAM")
print(reversed_string)
```

search

```python
@tool
def search(query: str, top_k: int = 3) -> str:
    """Search DuckDuckGo for the given query and return results."""
    wrapper = DuckDuckGoSearchAPIWrapper(region="vn-vi", max_results=top_k)
    search_obj = DuckDuckGoSearchResults(api_wrapper=wrapper, source="news")
    return search_obj.run(query)

results = search.invoke("Tin tức hôm nay")
print(results[:500])
```

multiply

```python
@tool
def multiply(a: int, b: int) -> int:
    """Multiply a and b."""
    return a * b
```

II.2.4. Bước 4 – Khai báo ChatOllama

Khai báo mô hình `qwen2.5:7b` qua Ollama. Tham số `temperature=0` đảm bảo đầu ra ổn định khi gọi tool:

Khởi tạo ChatOllama

```python
from langchain_ollama import ChatOllama

model_id = "qwen2.5:7b"

llm = ChatOllama(
    model=model_id,
    temperature=0,
    num_predict=512,
)
```

Test thử model trả lời câu hỏi đơn giản:

Test invoke

```python
llm.invoke("What is 2 multiplied by 3?")
```

II.2.5. Bước 5 – Bind tools và sinh tool call

`bind_tools([multiply])` gắn schema của `multiply` vào mỗi prompt gửi đến model. Khi gặp câu hỏi tính toán phức tạp, model sẽ *không tự tính sai* mà sinh ra `tool_calls` với tham số chính xác:

Bind tools + invoke

```
llm_with_tools = llm.bind_tools([multiply])

result = llm_with_tools.invoke("What is 7235  8421?")
result

result.tool_calls
```

Output `result.tool_calls` sẽ là một list các dict dạng:

Output tool$_{calls}$

```
[{
  "name": "multiply",
  "args": {"a": 7235, "b": 8421},
  "id": "call_xxx",
  "type": "tool_call"
}]
```

II.2.6. Bước 6 – Multi-tool routing

Khi đăng ký nhiều tool cùng lúc, model tự chọn tool phù hợp với từng câu hỏi:

Multi-tool routing

```
tools = [reverse_string, search, multiply]

model_with_tools = llm.bind_tools(tools)

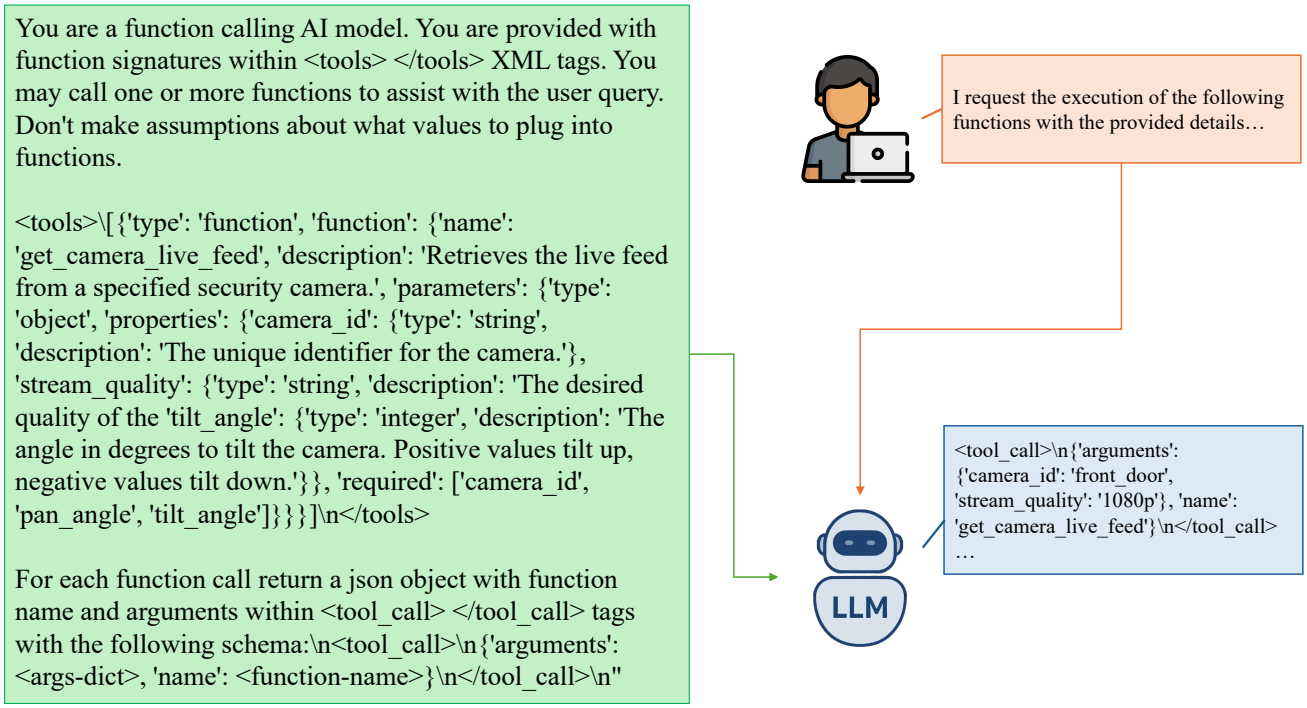
response = model_with_tools.invoke("Tin tức mới nhất hôm nay")
response

response.tool_calls
```

Với câu hỏi "Tin tức mới nhất hôm nay", model nhận ra cần gọi `search` thay vì `multiply` hay `reverse_string`. Cơ chế lựa chọn tool này hoàn toàn dựa vào docstring và type hint mà ta đã khai báo — đây là điểm mạnh của LangChain so với cách triển khai built-in thủ công.

III. Cải thiện khả năng gọi tool của mô hình với Instruction Fine-tuning

Một số mô hình lớn, đặc biệt là các mô hình có số lượng tham số thấp, việc hiểu và đưa ra các lời gọi hàm còn hạn chế do không được chú trọng cho phần này ở bước pre-training. Vì vậy, một cách có thể áp dụng để giải quyết vấn đề trên là sử dụng kỹ thuật **Instruction Fine-tuning** trên một bộ dữ liệu chứa các tác vụ kêu gọi mô hình lớn đưa ra các lời gọi hàm phù hợp với một yêu cầu và một danh sách hàm định nghĩa sẵn trong prompt.



Hình 13: Minh họa về bài toán huấn luyện mô hình với dữ liệu là các yêu cầu gọi hàm để giải quyết một yêu cầu từ người dùng.

Trong phần này, chúng ta sẽ đi qua các bước áp dụng Instruction Fine-tuning mô hình `Llama 3.-2 1B Instruct` trên bộ dữ liệu function calling `NousResearch/hermes-function-calling-v1`, sử dụng Unsloth (tăng tốc training) + LoRA (chỉ fine-tune một phần nhỏ tham số). Pipeline gồm 8 bước.

III.1. Bước 1 – Tải và import các thư viện cần thiết

Cài đặt thư viện

```
!pip install unsloth bitsandbytes peft
```

III.2. Bước 2 – Cấu hình môi trường (tránh lỗi Triton compile)

Trên GPU T4 / Colab, `torch.compile` đôi khi gây lỗi `InductorError SubprocException` do Triton subprocess compile thất bại. Disable dynamo và compile trước khi import các thư viện:

Disable torch.compile

```
# FIX: disable torch.compile / dynamo before any import
# (Triton subprocess compile fails on Colab T4 + PyTorch 2.5+)
import os
os.environ['TORCHDYNAMO_DISABLE'] = '1'
os.environ['TORCH_COMPILE_DISABLE'] = '1'
os.environ['UNSLOTH_DISABLE_AUTO_OPT'] = '1'

import torch
torch._dynamo.config.suppress_errors = True
torch._dynamo.config.disable = True
```

III.3. Bước 3 – Import thư viện

Imports

```
import unsloth
import os, torch, random, json
from datasets import load_dataset
from transformers import (
    TrainingArguments,
    set_seed
)
from trl import SFTTrainer, SFTConfig
from unsloth import FastLanguageModel


set_seed(59)
os.environ["CUDA_VISIBLE_DEVICES"] = "1"
```

III.4. Bước 4 – Khai báo mô hình

Khai báo mô hình pre-trained để thực hiện huấn luyện.
Ta dùng `FastLanguageModel.from_pretrained` của Unsloth với `load_in_4bit=True` để giảm bộ nhớ:

Load Llama 3.2 1B Instruct (Unsloth)

```
model_id = "meta-llama/Llama-3.2-1B-Instruct"
max_seq_length = 2048
model, tokenizer = FastLanguageModel.from_pretrained(
    model_id,
    max_seq_length = max_seq_length,
    load_in_4bit = True,
)
```

III.5. Bước 5 – Khai báo cài đặt và áp dụng LoRA vào mô hình

Để việc huấn luyện ít tốn kém hơn và vẫn đảm bảo tính hiệu quả, ta ứng dụng kỹ thuật LoRA. LoRA chỉ fine-tune ma trận hạng thấp ($A \in \mathbb{R}^{r\times d}$, $B \in \mathbb{R}^{d\times r}$) thay vì full weight, giảm số tham số trainable từ 1B xuống vài MB:

LoRA configuration

```
model = FastLanguageModel.get_peft_model(
    model,
    r = 16,
    lora_alpha = 32,
    lora_dropout = 0.05,
    target_modules = ["q_proj","k_proj","v_proj","o_proj",
                      "gate_proj","up_proj","down_proj"],
    bias = "none"
)
```

III.6. Bước 6 – Tải và xử lý bộ dữ liệu Hermes

Trong bài này, ta sử dụng bộ `hermes-function-calling-v1` từ NousResearch. Đây là bộ dữ liệu instruction với nội dung tool calling trên nhiều ngữ cảnh khác nhau. Vì bộ dữ liệu Hermes có format conversation hơi khác (role là `human`/`gpt`), ta cần chuyển về định dạng ChatML chuẩn của Llama (`user`/`assistant`):

Load + format dataset

```
dataset_id = "NousResearch/hermes-function-calling-v1"
raw_ds = load_dataset(dataset_id, split="train")

SYS_PROMPT = (
    "Cut-off knowledge: Dec 2023\n"
    "Today: 23 July 2024\n\n"
    "You are a helpful assistant with tool-calling capabilities."
)

def to_chat(example):
    msgs = [{"role": "system", "content": SYS_PROMPT}]
    role = {"system": "system", "human": "user", "gpt": "assistant"}
    for turn in example["conversations"]:
        msgs.append({"role": role[turn["from"]], "content": turn["value"]})
    chat_str = tokenizer.apply_chat_template(
        msgs,
        tokenize=False,
        add_generation_prompt=False
    )
    return {"text": chat_str}

ds = raw_ds.map(to_chat).remove_columns(
        [c for c in raw_ds.column_names if c != "text"]
)
```

Với bộ dữ liệu này, do sự đa dạng về mặt ngữ cảnh gọi tool, ta kỳ vọng rằng sau khi huấn luyện mô hình sẽ có thể cải thiện khả năng *in-context learning* với bất kỳ nội dung tool calling.

III.7. Bước 7 – Khai báo cài đặt huấn luyện và thực hiện huấn luyện

`TrainingArguments` với `optim="adamw_8bit"`, `torch_compile=False` để tránh lỗi Triton compile. `max_steps=60` cho demo nhanh; bỏ flag này nếu muốn huấn luyện đầy đủ một epoch:

TrainingArguments + SFTTrainer

```
training_args = TrainingArguments(
    output_dir = "./llama3-toolcall-lora",
    per_device_train_batch_size = 4,
    gradient_accumulation_steps = 4,
    num_train_epochs = 1,
    max_steps = 60,
    learning_rate = 2e-4,
    lr_scheduler_type = "cosine",
    bf16 = torch.cuda.is_bf16_supported(),
    optim = "adamw_8bit",
    torch_compile = False,
    logging_steps = 25,
```

```
    save_strategy = "epoch",
    push_to_hub = False,
)

sft_cfg = SFTConfig(dataset_text_field="text",
                    max_seq_length=max_seq_length)

trainer = SFTTrainer(
    model         = model,
    tokenizer     = tokenizer,
    train_dataset = ds,
    args          = training_args,
    **sft_cfg.to_dict(),
)
trainer.train()
```

III.8. Bước 8 – Lưu mô hình đã huấn luyện

Cuối cùng, ta lưu lại các thông tin mô hình sau huấn luyện để có thể tái sử dụng:

Save LoRA adapter

```
trainer.model.save_pretrained("./llama3-toolcall-lora")
tokenizer.save_pretrained("./llama3-toolcall-lora")
```

III.9. Bước 9 – Inference với mô hình đã fine-tune

Sau khi fine-tune, mô hình 1B sinh tool call đúng cú pháp JSON tốt hơn rõ rệt. Phần inference được chia thành **ba khối nhỏ**: (a) load lại mô hình với adapter LoRA đã lưu, (b) định nghĩa hàm `chat()` đóng gói vòng generate, (c) gọi `chat()` với một ví dụ. Tách như vậy giúp dễ tái sử dụng và debug.

(a) Load lại mô hình + LoRA adapter:

Load model + adapter

```
import torch
from unsloth import FastLanguageModel
from peft import PeftModel

device = "cuda" if torch.cuda.is_available() else "cpu"
max_seq_length = 2048

base_id = "meta-llama/Llama-3.2-1B-Instruct"
lora_dir = "./llama3-toolcall-lora"

model, tokenizer = FastLanguageModel.from_pretrained(
    base_id,
```

```
    max_seq_length = max_seq_length,
    load_in_4bit = True,
)
model = PeftModel.from_pretrained(model, lora_dir, is_trainable=False)
```

(b) Hàm `chat()` đóng gói vòng generate. Hàm nhận `user_msg` (full prompt theo Hermes format gồm tools schema + user query), apply chat template, sinh output và tách phần sau token `assistant`:

Hàm chat()

```
def chat(user_msg):
    prompt = tokenizer.apply_chat_template(
        [{"role": "user", "content": user_msg}],
        tokenize=False,
        add_generation_prompt=True,
    )
    inputs = tokenizer(prompt, return_tensors="pt").to("cuda")
    with torch.no_grad():
        out = model.generate(
            **inputs,
            max_new_tokens=max_seq_length,
            temperature=0.1,
        )
    decoded = tokenizer.decode(out[0], skip_special_tokens=True)
    return decoded.split("assistant")[-1].strip()
```

(c) Ví dụ chạy thử. Gọi `chat()` với một input dạng Hermes function-calling. Phần `<tools>-...</tools>` chứa JSON schema; phần sau `<|eot_id|>` là yêu cầu thực của user.

Run inference (rút gọn)

```
chat_msg = """
You are a function calling AI model. You are provided with function
signatures within <tools> </tools> XML tags. You may call one or more
functions to assist with the user query.
<tools>
[{"type": "function",
  "function": {"name": "get_camera_live_feed",
    "parameters": {"type": "object",
      "properties": {
        "camera_id": {"type": "string"},
        "stream_quality": {"type": "string",
                           "enum": ["720p", "1080p", "4k"]}},
      "required": ["camera_id"]}}}]
</tools>
For each function call return a json object with name and arguments
within <tool_call> </tool_call> tags.
<|eot_id|><|start_header_id|>user<|end_header_id|>
```

```
18
19 Retrieve the live feed from the "front_door" camera at 1080p quality.
20 """
21
22 response = chat(chat_msg)
23 print(response)
```

Output kỳ vọng (sau khi fine-tune) sẽ có dạng tool call hợp lệ:

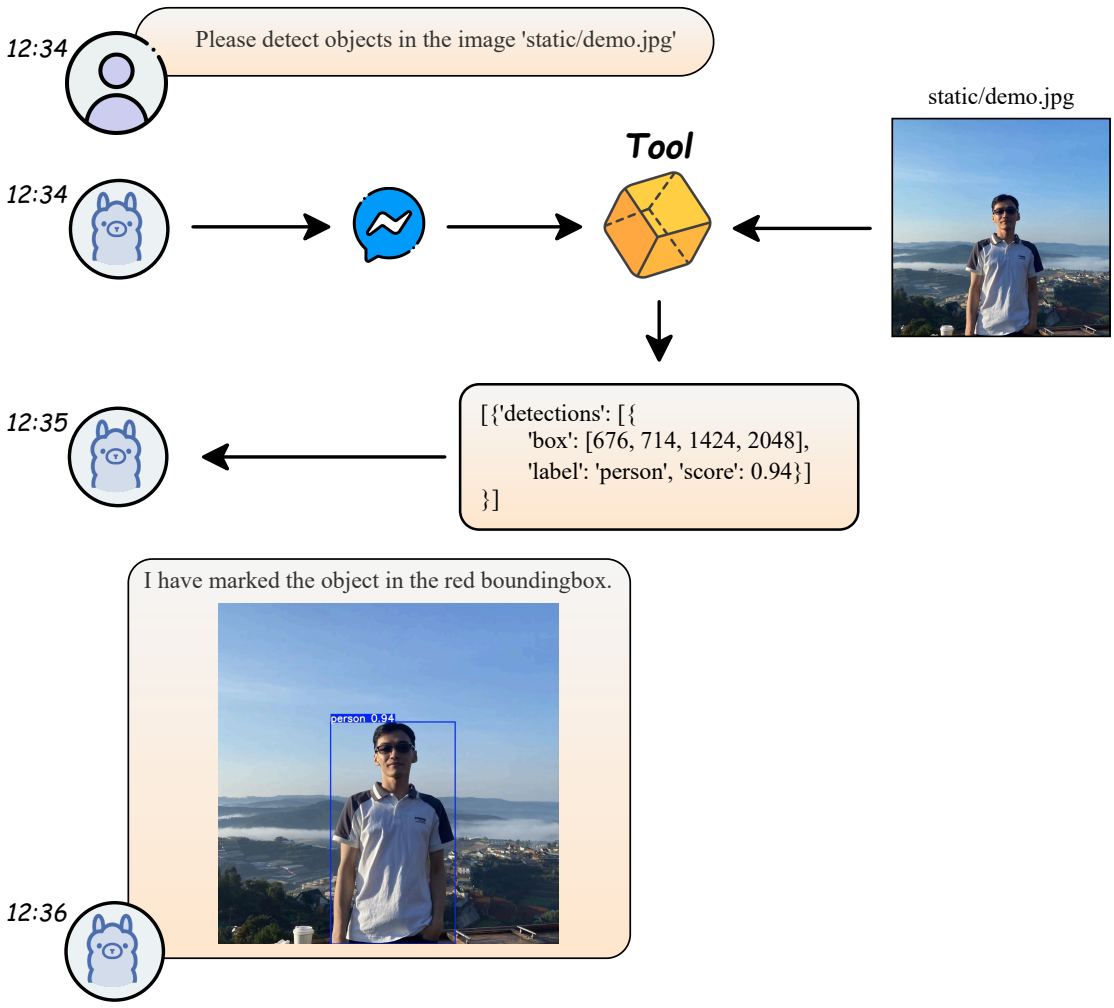
Sample output

```
1 <tool_call>
2 {"arguments": {"camera_id": "front_door", "stream_quality": "1080p"},
3  "name": "get_camera_live_feed"}
4 </tool_call>
```

IV. Các tác vụ xử lý ảnh thông qua Tool Calling

Trong phần này, chúng ta sẽ cùng tìm hiểu cách ứng dụng kỹ thuật Tool Calling vào mô hình lớn (chỉ xử lý text) để giải các bài toán liên quan đến thị giác máy tính. Cụ thể, ta sẽ xây dựng chương trình có khả năng thực hiện các tác vụ **Object Detection** và **Object Segmentation** trên một ảnh. Như vậy, Input/Output của bài này:

- **Input**: một ảnh có sẵn trong máy tính và một câu mô tả bài toán (object detection hoặc object segmentation) cho mô hình.
- **Output**: giải quyết thành công yêu cầu đầu vào, có thể bao gồm trả về toạ độ bounding box hoặc mask của vật thể trong ảnh.



Hình 14: Minh họa bài toán gọi tool là các mô hình vision để giải quyết các tác vụ thị giác máy tính với LLMs Tool Calling.

Chúng ta cài đặt theo các bước sau.

IV.1. Bước 1 – Tải các thư viện cần thiết

Phần này cần cài thêm thư viện Ultralytics để sử dụng mô hình YOLO v11:

Pip install

```
!pip install -q ultralytics opencv-python transformers torch vllm unsloth
```

IV.2. Bước 2 – Import các thư viện và chọn device

Imports

```
import os
import torch
import json
import ast
import sys
from transformers import SamModel, SamProcessor
from transformers import AutoModelForCausalLM, AutoTokenizer
from transformers import GenerationConfig
from PIL import Image
from ultralytics import YOLO


os.environ["CUDA_VISIBLE_DEVICES"] = "1"
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
```

IV.3. Bước 3 – Định nghĩa Tools (detection + segmentation)

Tới đây, ta định nghĩa hai tool dưới dạng các hàm Python. Mỗi hàm có chức năng giải quyết một bài toán thị giác máy tính cụ thể: detection bằng YOLOv11 và segmentation bằng SAM (Segment Anything Model).

Hàm detection và segmentation

Hàm `run_detection` gói toàn bộ quy trình YOLO v11 dưới một giao diện thống nhất: nhận đường dẫn ảnh, suy luận, chuyển kết quả sang định dạng JSON. Hàm `run_segmentation` tương tự, trích xuất binary masks từ SAM:

run_detection + run_segmentation

```
detection_model_id = "yolo11n.pt"
detection_model = YOLO(detection_model_id)
def run_detection(image_path: str, is_visualize: bool = False):
```

```python
    """YOLOv11: return list of {box, label, score} for a single image."""
    results = detection_model(image_path)
    r = results[0]

    detections = []
    for box in r.boxes:
        # box.xyxy is a 14 tensor, box.conf is a 1-element tensor, box.cls likewise
        coords  = box.xyxy.cpu().numpy().flatten().tolist()
        score   = float(box.conf.cpu().numpy().item())
        cls_id  = int(box.cls.cpu().numpy().item())
        detections.append({
            "box": coords,
            "label": r.names[cls_id],
            "score": score,
        })

    if is_visualize:
        r.save()
        r.show()

    return {"detections": detections}

segmentation_model_id = "facebook/sam-vit-base"
sam_processor = SamProcessor.from_pretrained(segmentation_model_id)
sam_model = SamModel.from_pretrained(segmentation_model_id)
def run_segmentation(image_path: str):
    """SAM: return binary masks as nested lists"""
    img = Image.open(image_path).convert("RGB")
    inputs = sam_processor(images=img, return_tensors="pt")
    outputs = sam_model(**inputs)

    masks = outputs.pred_masks.squeeze(0).cpu().detach().numpy().tolist()
    return {"masks": masks}

tool_functions = {
    "run_detection": run_detection,
    "run_segmentation": run_segmentation
}
```

Test riêng từng tool:

Test riêng từng tool

```python
run_segmentation('cat.jpg')

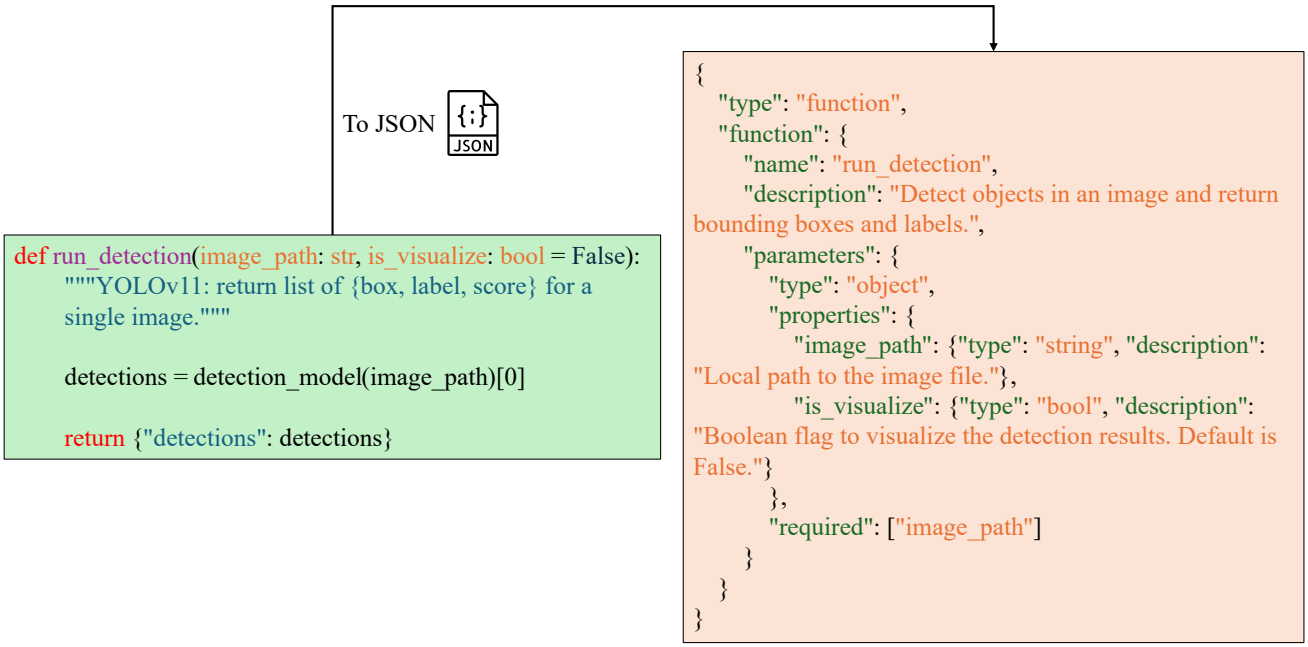
run_detection("cat.jpg", is_visualize=True)
```

IV.4. Bước 4 – Định nghĩa danh sách tools (JSON Schema)

Để giúp mô hình nắm được danh sách các hàm có thể sử dụng, ta định nghĩa một list JSON chuẩn theo OpenAI function calling format: `type=function`, `function.name`, `function.parameters` (với `type=object`, `properties`, `required`):

functions_lst (JSON Schema)

```
functions_lst = [
    {
        "type": "function",
        "function": {
            "name": "run_detection",
            "description": "Detect objects in an image and return bounding boxes and labels.",
            "parameters": {
                "type": "object",
                "properties": {
                    "image_path": {"type": "string", "description": "Local path to the image file."},
                    "is_visualize": {"type": "bool", "description": "Boolean flag to visualize the detection results. Default is False."}
                },
                "required": ["image_path"]
            }
        }
    },
    {
        "type": "function",
        "function": {
            "name": "run_segmentation",
            "description": "Segment objects in an image and return binary masks.",
            "parameters": {
                "type": "object",
                "properties": {
                    "image_path": {"type": "string", "description": "Local path to the image file."}
                },
                "required": ["image_path"]
            }
        }
    }
]
```



Hình 15: Minh hoạ việc đưa thông tin của hàm thành dạng JSON để mô hình có thể hiểu và sử dụng.

IV.5. Bước 5 – Định nghĩa system prompt

Khi đã có danh sách hàm dưới dạng JSON, ta định nghĩa system prompt để mô hình hiểu nhiệm vụ và format output. System prompt yêu cầu output `[func(param=value, ...)]` chuẩn xác:

SYSTEM_PROMPT

```
SYSTEM_PROMPT = """
You are an expert in composing functions. You are given a question and a set of possible functions.
Based on the question, you will need to make one or more function/tool calls to achieve the purpose.
If none of the function can be used, point it out. If the given question lacks the parameters required by the function,
also point it out. You should only return the function call in tools call sections.

If you decide to invoke any of the function(s), you MUST put it in the format of [func_name1(params_name1=params_value1, params_name2=params_value2...), func_name2(params)]\n
You SHOULD NOT include any other text in the response.

Here is a list of functions in JSON format that you can invoke.\n\n{functions}\n""".format(functions=functions_lst)
```

IV.6. Bước 6 – Khai báo mô hình LLM

Tương tự các bài trước, ta dùng `Llama 3.2 1B Instruct`:

HF login + load Llama 3.2 1B

```
from huggingface_hub import login
login(token='hf_xxx')

model_id = "meta-llama/Llama-3.2-1B-Instruct"
llm = AutoModelForCausalLM.from_pretrained(model_id,
                                           device_map=device,
                                           torch_dtype=torch.bfloat16)
tokenizer = AutoTokenizer.from_pretrained(model_id)
```

IV.7. Bước 7 – Pipeline thực thi tool call

Cuối cùng, ta hoàn tất pipeline gọi tool. Có một thay đổi quan trọng so với phiên bản tài liệu cũ: **parser được cải tiến để xử lý các trường hợp mô hình 1B sinh output sai format**. Cụ thể, `Llama 3.2 1B` chưa fine-tune có thể trả về:

- `[run_segmentation(image_path='cat.jpg')]` — format đúng, theo system prompt.
- `(run_segmentation(image_path='cat.jpg'))` — sai dấu ngoặc, dùng `(...)` thay vì `[...]`.
- `(run_segmentation(image_path='cat.jpg'))assistant\n\n[]` — thừa text + có `[]` rỗng.
- `Sure, I'll call run_segmentation('cat.jpg').` — tool call lẫn trong văn bản tự nhiên.

Hàm `extract_tool_calls` dưới đây dùng strategy *robust*: scan toàn bộ output text bằng regex, lấy mọi pattern `fn_name(...)` thuộc whitelist (`run_detection`, `run_segmentation`). Cách này chấp nhận tất cả các biến thể trên, không phụ thuộc vào dấu wrapper:

Pipeline với fallback parser

```
messages = [
    {"role": "system", "content": SYSTEM_PROMPT},
    {"role": "user",   "content": "Please segment objects in the image 'cat.jpg'."},
]

bos_id = tokenizer.bos_token_id if tokenizer.bos_token_id is not None else tokenizer.
                                                 eos_token_id

generation_config = GenerationConfig(
    max_new_tokens=512,
    temperature=0.5,
    top_p=0.98,
```

```
    top_k=50,
    repetition_penalty=1.2,
    do_sample=True,
    num_return_sequences=1,
    bos_token_id=bos_id,
    eos_token_id=tokenizer.eos_token_id,
    pad_token_id=tokenizer.eos_token_id,
)

prompt_text = tokenizer.apply_chat_template(
    messages,
    tokenize=False,
    add_generation_prompt=True,
)

inputs = tokenizer(
    prompt_text,
    return_tensors="pt",
    return_attention_mask=True,
).to(device)

outputs = llm.generate(
    input_ids=inputs.input_ids,
    attention_mask=inputs.attention_mask,
    generation_config=generation_config,
)

import re

# Decode chỉ phần model sinh ra (bỏ phần prompt)
input_len = inputs.input_ids.shape[1]
generated_ids = outputs[0][input_len:]
raw = tokenizer.decode(generated_ids, skip_special_tokens=True).strip()
print("Raw output:", repr(raw))

# Trích xuất tool calls  model nhỏ (1B) có thể trả [..], (..), `[]assistant`, hoặc raw func(..)
# Strategy đơn giản: scan toàn bộ text tìm `fn_name(...)` ở danh sách allowed,
# không phụ thuộc vào dấu wrapper [], (). Tránh bug `[]` empty match thắng nội dung thực.
def extract_tool_calls(raw_text, allowed_fns):
    func_pattern = re.compile(
        r"\b([a-zA-Z_][a-zA-Z0-9_]*)\(([^()]*(?:\([^()]*\)[^()]*)*)\)"
    )
    candidates = []
    for fm in func_pattern.finditer(raw_text):
        if fm.group(1) in allowed_fns:
            candidates.append(fm.group(0))
    if candidates:
        return "[" + ", ".join(candidates) + "]"
    return None

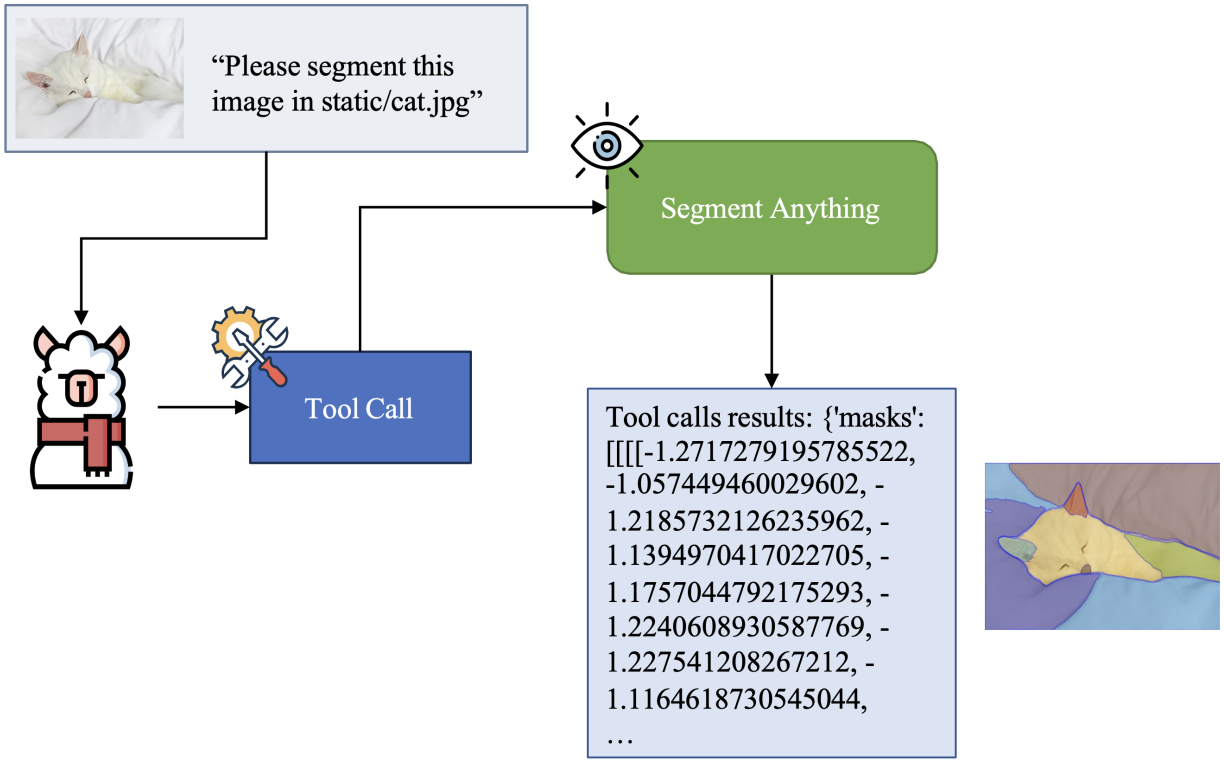
tool_calls_lst_str = extract_tool_calls(raw, tool_functions)

```

```python
if not tool_calls_lst_str:
    print(" Model không sinh ra tool call hợp lệ.")
    tool_calls_result = []
else:
    print("Parsed:", tool_calls_lst_str)
    try:
        tree = ast.parse(tool_calls_lst_str, mode='eval')
        if not isinstance(tree.body, ast.List):
            raise ValueError(f"Expected list, got {type(tree.body).__name__}")
        call_nodes = tree.body.elts

        tool_calls_result = []
        for call in call_nodes:
            function_name = call.func.id
            parameters = {
                kw.arg: ast.literal_eval(kw.value)
                for kw in call.keywords
            }
            result = tool_functions[function_name](**parameters)
            tool_calls_result.append({"function": function_name, "result": result})
    except (SyntaxError, ValueError, AttributeError) as e:
        print(f" Cannot parse: {e}")
        tool_calls_result = []

print("Tool calls results:", tool_calls_result)
```



Hình 16: Pipeline vision tool calling: user gửi prompt “Please segment this image in static/cat.jpg”, LLM phân tích yêu cầu và gọi tool `run_segmentation`, kết quả `masks` được trả về và trực quan hoá.

Khi thực thi đoạn code trên, ta sẽ nhận được kết quả các lời gọi hàm để giải quyết bài toán đầu vào. Tuỳ thuộc vào cách định nghĩa hàm mà ta sẽ có các chức năng khác nhau để sử dụng. Ví dụ, với input “Please segment objects in the image cat.jpg”, model gọi `run_segmentation('cat.jpg')` và nhận về masks; nếu input là “Detect objects in cat.jpg”, model gọi `run_detection('cat.jpg')` và nhận về bounding boxes.

V. Câu hỏi trắc nghiệm

1. (Khái niệm – Function/Tool Calling là gì)

Function Calling trong LLMs được hiểu là phương thức nào dưới đây?

A. Kỹ thuật cho phép LLM tự động sinh thêm hàm nội bộ để mở rộng khả năng suy luận.

B. Cơ chế tích hợp LLM với các hàm/công cụ bên ngoài để thực hiện các tác vụ vượt quá giới hạn của bản thân mô hình.

C. Phương pháp huấn luyện LLM bằng cách chèn các hàm toán học phức tạp trực tiếp vào kiến trúc Transformer.

D. Hình thức tinh chỉnh mô hình ngôn ngữ lớn thông qua việc nhúng mã Python vào prompt.

2. (Khái niệm – Mục tiêu của Function Calling)

Mục tiêu chính của Function Calling trong khuôn khổ LLMs là gì?

A. Tăng tốc độ xử lý token và giảm độ trễ phản hồi.

B. Bổ sung thông tin thời gian thực và dữ liệu đa phương thức mà mô hình chưa được đào tạo.

C. Nén kích thước mô hình để triển khai trên các thiết bị có tài nguyên hạn chế.

D. Cải thiện độ chính xác ngữ pháp trong quá trình sinh văn bản.

3. (Implementation – Output khi LLM gọi hàm)

Khi LLM quyết định thực hiện một lời gọi hàm (function call), nó sẽ xuất ra gì để client có thể thực thi hàm tương ứng?

A. Một đối tượng JSON chứa tên hàm cùng tham số kèm kiểu dữ liệu.

B. Một file script Python hoàn chỉnh bao gồm hàm và các thư viện liên quan.

C. Một URI trỏ đến endpoint của hàm được định nghĩa trên server.

D. Một đoạn văn bản mô tả chức năng và cách sử dụng hàm.

4. (Khái niệm – Hạn chế của LLM mà Tool Calling khắc phục)

Hạn chế nổi bật của LLM phiên bản trước mà Function Calling hướng tới khắc phục là gì?

A. Khả năng sinh văn bản không vượt quá 512 token.

B. Không thể truy xuất hoặc khai thác thông tin thời gian thực.

C. Không hỗ trợ xử lý đa ngôn ngữ trong cùng một đầu vào.

D. Không thể phân tích dữ liệu hình ảnh hoặc âm thanh đầu vào.

5. (Implementation – Vai trò của bộ dữ liệu Hermes)

Dataset `NousResearch/hermes-function-calling-v1` chủ yếu được sử dụng để:

A. Đánh giá khả năng dịch ngôn ngữ qua API function calling.

B. Huấn luyện LLM cải thiện kỹ năng tóm tắt văn bản.

C. Fine-tuning LLM để thực hành Function Calling.

D. Triển khai chatbot đa kênh với giao diện người dùng phong phú.

6. (Implementation – Mục tiêu khi fine-tune cho Tool Calling)

Khi fine-tuning Llama 3.2 trên bộ dữ liệu Function Calling, mục tiêu chính là gì?

A. Giảm độ trễ khi triển khai trên môi trường sản xuất.

B. Nâng cao khả năng sinh mã JSON hoặc lệnh gọi hàm với cú pháp chính xác.

C. Tăng cường năng lực sáng tác thơ văn, tiểu thuyết.

D. Cải thiện khả năng tổng hợp và sinh ảnh AI.

7. (Implementation – Hành vi của client sau khi nhận tool call)

Sau khi client nhận được phản hồi JSON chứa lệnh gọi hàm từ LLM, bước tiếp theo thường là gì?

A. Gửi JSON đó ngược lại cho LLM để xác nhận cú pháp.

B. Phân tích (parse) JSON rồi gọi hàm tương ứng với tên và tham số đã chỉ định.

C. Hiển thị JSON cho người dùng để họ quyết định hàm nào được thực thi.

D. Lưu JSON vào cơ sở dữ liệu để sử dụng cho quá trình huấn luyện tiếp theo.

8. (Khái niệm – Function schema)

Trong ngữ cảnh Function Calling, định nghĩa "function schema" thường được biểu diễn dưới dạng nào sau đây?

A. Tập tin YAML schema mô tả tham số và kiểu dữ liệu.

B. JSON Schema định nghĩa tên hàm, tham số và kiểu dữ liệu tương ứng.

C. Tệp ProtoBuf (`*.proto`) dùng để serialize tham số hàm.

D. Định nghĩa XML tương tác qua giao thức SOAP API.

9. (Implementation – Khi mô hình gọi hàm qua JSON Schema)

Trong ngữ cảnh Function Calling, khi mô hình gọi một hàm thông qua JSON Schema, điều gì sẽ xảy ra?

A. Mô hình xác định các tham số cần thiết và đảm bảo các giá trị đầu vào hợp lệ.

B. Mô hình trực tiếp xử lý dữ liệu đầu vào và trả về kết quả mà không cần gọi hàm.

C. Mô hình lưu trữ kết quả trả về trong cơ sở dữ liệu để huấn luyện lại sau.

D. Mô hình yêu cầu người dùng cung cấp các giá trị đầu vào theo định dạng XML.

10. (Implementation – Yếu tố quan trọng nhất khi soạn prompt)

Khi soạn prompt cho Function Calling, yếu tố quan trọng nhất để đảm bảo LLM sinh ra lệnh gọi hàm hợp lệ là gì?

A. Mô tả chi tiết thuật toán bên trong hàm.

B. Định nghĩa rõ ràng tên hàm và kiểu dữ liệu của từng tham số.

C. Cung cấp ví dụ về file đầu vào và đầu ra mong muốn.

D. Mô tả môi trường thực thi (environment) của hàm.

VI. Tài liệu tham khảo

[1] OpenAI, *Function calling guide*, https://platform.openai.com/docs/guides/function-calling, 2024.

[2] JSON Schema Org, *Json schema specification*, https://json-schema.org/, 2024.

[3] OpenAI, *How to call functions with chat models*, https://github.com/openai/openai-cookbook/blob/main/examples/How_to_call_functions_with_chat_models.ipynb, 2024.

[4] Meta AI, *Llama 3 prompt format and chat template*, https://www.llama.com/docs/model-cards-and-prompt-formats/llama3_2, 2024.

[5] LangChain, *Tool calling — langchain documentation*, https://python.langchain.com/docs/concepts/tool_calling/, 2024.

[6] NousResearch, *Hermes function calling v1 dataset*, https://huggingface.co/datasets/NousResearch/hermes-function-calling-v1, 2024.

[7] D. Han et al., *Unsloth: 2x faster llm fine-tuning*, https://github.com/unslothai/unsloth, 2024.

[8] E. J. Hu et al., *Lora: Low-rank adaptation of large language models*, https://arxiv.org/abs/2106.09685, 2021.

[9] Ultralytics, *Yolov11 documentation*, https://docs.ultralytics.com/models/yolo11/, 2024.

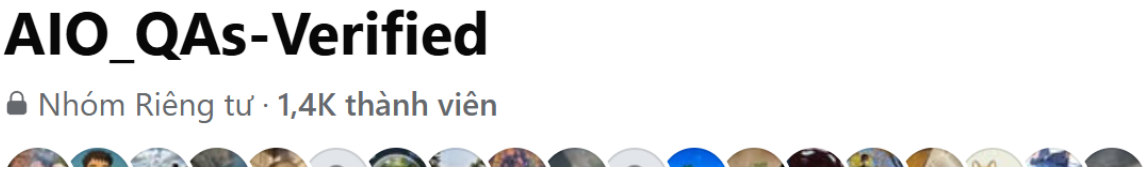
[10] A. Kirillov et al., *Segment anything*, https://arxiv.org/abs/2304.02643, 2023.

[11] Ollama, *Ollama — run llama and other llms locally*, https://ollama.com/, 2024.

[12] Qwen Team, *Qwen2.5 technical report*, https://arxiv.org/abs/2412.15115, 2024.

Phụ lục

1. **Hint:** Các file code gợi ý và dữ liệu nếu có được lưu trong thư mục có thể được tải tại đây.
2. **Solution:** Các file code cài đặt hoàn chỉnh và phần trả lời nội dung trắc nghiệm có thể được tải tại đây (Lưu ý: Sáng thứ 3 khi hết deadline phần project, ad mới copy các nội dung bài giải nêu trên vào đường dẫn).
3. **Q&A:** Bạn có thể đặt thêm câu hỏi về nội dung bài đọc trong group Facebook hỏi đáp tại đây. Tất cả câu hỏi sẽ được trả lời trong vòng tối đa 4 giờ.



Hình 17: Hình ảnh group facebook AIO Q&A

4. **Rubric:**

Phần	Kiến Thức	Đánh Giá
1.	- Hiểu khái niệm Tool Calling, Function Calling và phân biệt với AI Agent - Nắm được vòng đời 4 bước (Tool Creation, Tool Binding, Tool Calling, Tool Invocation) - Biết về các loại tool (Built-in, JSON-based, Custom)	- Phân biệt được khi nào model trả lời tự nhiên, khi nào sinh tool call - Đọc và hiểu được output JSON / list-format mà model sinh ra - Nhận diện được pattern System Prompt của Llama 3 (`<|python_tag|>`, `<|eom_id|>`)
2.	- Hiểu cơ chế `@tool` decorator + `bind_tools()` trong LangChain - Sử dụng được `ChatOllama` với mô hình `qwen2.5:7b` cho native tool calling - Định nghĩa được LLM Router để phân nhánh dựa trên ngữ cảnh	- Cài đặt và chạy được pipeline tool calling end-to-end - Multi-tool routing: model tự chọn tool phù hợp giữa nhiều tool đã đăng ký - Debug được các lỗi thường gặp (DDGS rename, init_chat_model bind_tools, apply_chat_template conflict)
3.	- Hiểu được mục tiêu Instruction Fine-tuning cho tool calling - Nắm được kỹ thuật LoRA và lý do chỉ fine-tune một phần nhỏ tham số - Hiểu cách orchestrate Vision models (YOLOv11 + SAM) qua tool call	- Train được LoRA adapter trên Hermes dataset, lưu và load lại - Inference sau fine-tune sinh được tool call đúng cú pháp JSON - Định nghĩa được tool wrapper cho YOLO và SAM